SOURCE-GROUNDED ARCHITECTURE REPORT

Pi Agent Harness

Architecture, loop flow, extension points, and a practical path from a minimalist coding agent to a powerful general-purpose harness

Central conclusion: Pi already provides the core agent loop and a capable session harness. The most reliable expansion strategy is to preserve that core, add policy and verification through extensions, and introduce an SDK supervisor only when multiple agents, background work, scheduling, or shared durable memory are required.

Prepared for Pedro | 21 July 2026 | Europe/Lisbon

Primary source: official Pi GitHub repository

Executive summary

Pi is a genuine agent harness rather than merely an LLM client with four command functions. Its minimalism is a product philosophy: it deliberately omits opinionated implementations of subagents, workflow planning, MCP, permissions, and long-term semantic memory. Underneath, it already contains a sophisticated model/tool loop, streaming state, session persistence, branching, retries, compaction, resource discovery, and an extensive extension surface.

The practical implication is important: changing `agent-loop.ts` is not the first step toward a stronger harness. Most enhancements belong either in a TypeScript extension or in a supervisor application built with the SDK.

> Recommended architecture:
> 1. Extensions for tools, policies, context injection, lifecycle hooks, verification, and UI.
> 2. An SDK supervisor for multiple sessions, background jobs, shared memory, scheduling, and workflow orchestration.
> 3. A sandbox or container before granting unattended agents broad filesystem, shell, credential, or network access.

Scope and source basis

This report traces the latest upstream architecture requested by the user. At research time, the former `badlogic/pi-mono` URL redirected to `earendil-works/pi`, and the current package namespace was `@earendil-works`. Exact implementation claims below are based on the official source and documentation.

The supplied video - Harness Engineering Masterclass: Technical Deep Dive on how to build Agentic Systems - provides the wider harness framing. Its complete transcript was not available through the research interface, so the Pi-specific analysis does not rely on an inferred transcript.

1. High-level architecture

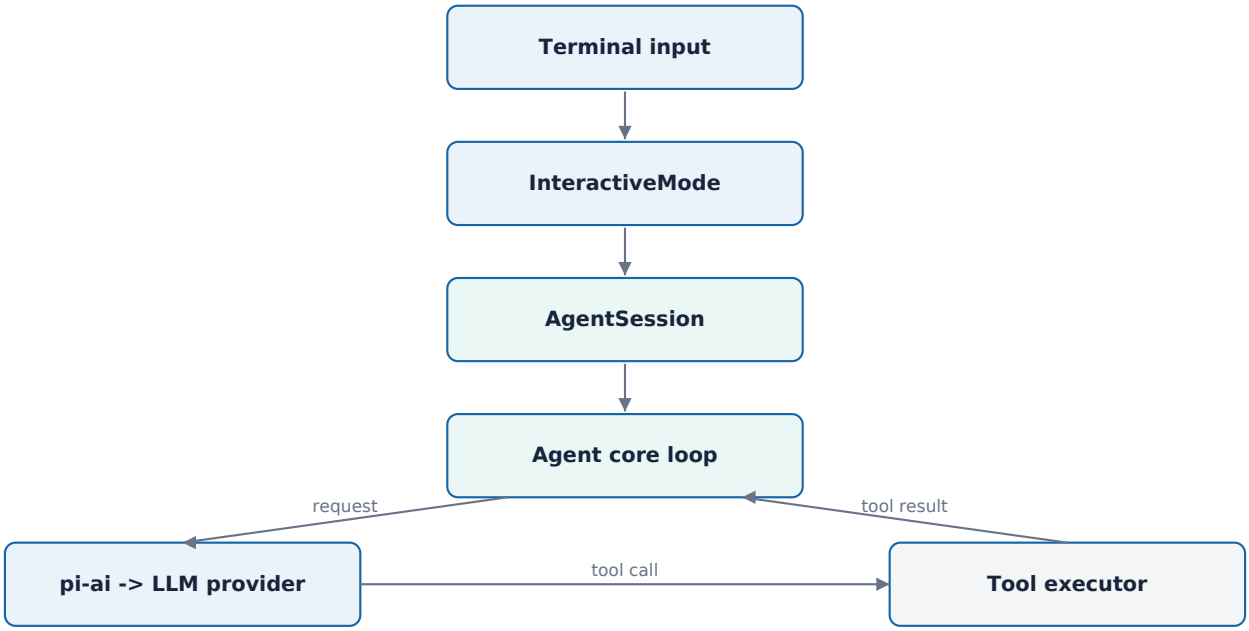


Figure 1. The model requests actions; the harness validates and executes them. The model never invokes the operating system directly.

A model response has two useful outcomes: ordinary text, which can finish the work, or a structured tool call, which asks the harness to perform an action. Pi validates that request, executes ordinary TypeScript/Node.js code, turns the result into a `toolResult` message, and calls the model again.

Three different loops

Concept	Meaning
Interactive loop	Waits indefinitely for the next terminal input.
Agent run	All work triggered by one submitted prompt, including multiple model calls and tools.
Turn	Exactly one LLM request, its streamed response, and the tool batch requested by that response.

One user prompt can therefore produce several turns:

Your prompt
- Turn 1: model requests `read`
- Turn 2: model requests `edit`
- Turn 3: model requests `bash` to run tests
- Turn 4: model gives a final answer

Agent run ends; Pi returns to the interactive input loop.

2. Package architecture

Package	Responsibility
pi-coding-agent	Complete product: CLI, TUI integration, sessions, extensions, skills, prompts, built-in tools, compaction, and run modes.
pi-agent-core	Stateful agent object and deterministic model/tool loop.
pi-ai	Provider-independent LLM API, streaming, authentication, tool schemas, token usage, and cost data.
pi-tui	Terminal rendering, editor components, keyboard input, and interactive display.

> `pi-coding-agent` is the ready-made harness; `pi-agent-core` is its engine and loop; `pi-ai` is the provider adapter; and `pi-tui` is the terminal dashboard.

Official package overview: Pi repository README and pi-agent-core documentation.

3. Mid-level function trace

Stage	Main location	Responsibility
1. CLI/runtime	packages/coding-agent/src/main.ts	Parses arguments, resolves trust and resources, creates or resumes a session, selects the run mode, and constructs AgentSessionRuntime.
2. Terminal	modes/interactive/interactive-mode.ts	Waits for input and calls session.prompt(userInput).
3. Session	core/agent-session.ts	Expands commands, skills and templates; checks auth and compaction; injects extension context; then calls the Agent.
4. Agent	packages/agent/src/agent.ts	Owns live state and lifecycle, snapshots context/configuration, and invokes runAgentLoop().
5. Core loop	packages/agent/src/agent-loop.ts	Streams the model response, detects tool calls, executes tools, appends results, drains queues, and decides whether to continue.
6. Provider	packages/ai	Normalizes the request and streams events from the selected model provider.
7. Return path	AgentSession + InteractiveMode	Persists messages, emits extension events, handles retry/compaction, and renders the result.

Source entry points: main.ts, interactive-mode.ts, agent-session.ts, agent.ts, and agent-loop.ts.

3.1 Prompt preparation in AgentSession

- Detect and execute an extension slash command.
- Emit the extension input event, allowing the input to be transformed or handled.
- Expand /skill:name and file-based prompt templates.
- If an agent run is active, queue the message as steering or follow-up work.
- Validate the selected model and authentication.
- Check whether the previous context should be compacted.
- Create the user message and inject pending custom context messages.
- Emit before_agent_start, allowing extensions to add messages or replace the effective system prompt.
- Call the underlying Agent and manage post-run retry, compaction, and continuation.

3.2 Agent state

The `Agent` maintains the system prompt, model, thinking level, active tools, transcript messages, streaming status, partial assistant message, pending tool-call IDs, and the most recent error. `Agent.prompt()` prevents overlapping runs, creates an abort controller, snapshots the context, and invokes `runAgentLoop()`.

3.3 Simplified core loop

```
runLoop(context):
  pending = getSteeringMessages()

  while true:                        # follow-up loop
    continueForTools = true

    while continueForTools or pending:
      injectPendingMessages(pending)
      assistant = streamAssistantResponse(context)

      if assistant.error or assistant.aborted:
        stop

      toolCalls = extractToolCalls(assistant)
      continueForTools = false

      if toolCalls:
        results = executeToolCalls(toolCalls)
        appendToolResults(results)
        continueForTools = not results.terminate

      emitTurnEnd()
      context = prepareNextTurn(context)

      if shouldStopAfterTurn(context):
        stop

      pending = getSteeringMessages()

    pending = getFollowUpMessages()
    if not pending:
      break

  emitAgentEnd()
```

Continuation and stop conditions

The run continues when...	The run stops when...
The model requested one or more tools.	The model returns without tool calls and no queue remains.
A steering message is waiting.	The provider response errors or is aborted.
A follow-up message is waiting.	shouldStopAfterTurn requests a graceful stop.
AgentSession triggers continuation after retry or compaction.	Every finalized tool result in the batch requests terminate=true.

4. Context sent on every model turn

Before each LLM request, Pi takes its flexible internal transcript and creates a provider-compatible context:

> `AgentMessage[]` -> optional `transformContext()` -> required `convertToLlm()` -> provider messages -> LLM

The effective request includes the system prompt, effective conversation history, active tool definitions and schemas, and current model/reasoning options. Pi therefore resends the effective context on each model call. It does not depend on the remote model remembering a previous API request. The context

may first be pruned, transformed, or compacted.

The message transformation path is documented in pi-agent-core message flow.

4.1 System-prompt composition

- Pi's base coding-agent instructions.
- Active tool descriptions and tool-specific guidelines.
- Current working directory.
- Project and global AGENTS.md context files.
- Available skill summaries.
- Custom or appended system prompts.
- Per-run extension modifications.

See system-prompt.ts. Skills mainly provide procedures and context; extensions execute code and can alter harness behaviour.

5. Tool execution

Pi activates four tools by default: `read`, `write`, `edit`, and `bash`. The model receives their definitions and uses structured tool calls to request actions.

1. Extract structured toolCall blocks from the assistant message.
2. Find each registered tool by name.
3. Validate and normalize arguments against its schema.
4. Emit tool_execution_start.
5. Run beforeToolCall, which can block the operation.
6. Execute the tool and optionally stream progress updates.
7. Run afterToolCall, which can audit or replace the result.
8. Emit tool_execution_end.
9. Create a toolResult transcript message.
10. Call the model again with the updated context.

Tool calls execute in parallel by default. A tool may declare sequential execution; if one call in a batch targets a sequential tool, the entire batch executes sequentially. A thrown tool error is caught and returned to the model as an error result so the model can recover.

Sources: default tools and Pi philosophy and tool event sequence and execution modes.

6. Session layer around the loop

The core loop is deliberately generic. `AgentSession` turns it into a practical coding harness by persisting messages, forwarding events, tracking usage, handling model errors, retrying, compacting context, and supporting branching and resumption.

Already present	Not equivalent to long-term assistant memory
Durable JSONL transcript	No extracted user profile or fact store
Resume, tree navigation, fork and clone	No semantic retrieval across unrelated sessions
Token/cost accounting	No durable task database or scheduler

Already present	Not equivalent to long-term assistant memory
Automatic and manual compaction	No automatic lesson extraction and reuse

Sessions are JSONL trees. Each entry has an ID and parent ID, enabling multiple branches in one file. Compaction is lossy for the model-visible context, but full history remains available in the session file.

See sessions, branching, and compaction.

6.1 Steering versus follow-up

Queue	Delivery point
steer()	After the current assistant turn and its tool calls finish, before the next model request.
followUp()	Only when the agent would otherwise finish all current work.

Steering is therefore not a forceful mid-command interruption. It changes the instructions at the next safe turn boundary. This is an important built-in orchestration primitive.

7. Pi against the wider harness model

The supplied video frames a harness as the complete operating environment around the model: instructions, context, tools, execution, state, orchestration, safety, verification, and learning. Pi's present coverage can be summarized as follows.

Harness primitive	Pi's current position
Instructions	Strong: system prompt, AGENTS.md, prompt templates, append/override mechanisms.
Context delivery	Good: context files, custom messages, skills, read tool, extension injection.
Context management	Good: conversion hooks, compaction, branching, per-turn preparation.
Tool interface	Strong: schemas, validation, hooks, streaming updates, parallel/sequential execution.
Execution environment	Weak by default: tools inherit the Pi process user's permissions.
Durable state	Good session history; limited cross-session semantic memory and task state.
Orchestration	Good single-agent loop and queues; no built-in workflow graph or scheduler.
Subagents	Intentionally absent; can be built through extensions, processes, or SDK sessions.
Skills/procedures	Strong, standards-based, and progressively loaded.
Verification/observability	Rich lifecycle events, tokens, costs, and transcripts; no mandatory quality gates.
Learning loop	Not built in; requires memory extraction, evaluations, and policy updates.

> Security distinction: Project trust determines whether project settings and code extensions are loaded. It is not a general permission boundary around shell commands, files, network access, or credentials.

Pi's repository explicitly states that Pi runs with the permissions of its launching process and recommends sandboxing or containerization for stronger boundaries. See Permissions and Containerization.

8. Choosing between an extension and the SDK

Use an extension for...	Use an SDK supervisor for...
New LLM-callable tools	Several independent or concurrent AgentSession instances
Permission gates and path policies	Coordinator/worker/evaluator agent structures
Input transformation and context injection	Background jobs, schedules, cancellation and recovery
Tool-call auditing and result filtering	Shared memory and task state across sessions
Verification after tools or turns	Web, mobile, Telegram, voice or service interfaces
Commands, shortcuts and TUI widgets	Workflow state machines and human approval systems
Session lifecycle event handling	Different models assigned to different roles

Useful extension events include input, before_agent_start, agent_start/end, turn_start/end, message_start/update/end, tool_call, tool_result, session events, and compaction events. Extensions can also register tools, commands, shortcuts, flags, providers, and UI components.

Sources: Extension API types and SDK documentation.

9. Recommended expansion roadmap

1 Harness observability extension
Record prompts, turns, tools, failures, duration, token usage, retries, and completion status. Reliable diagnosis must precede complex orchestration.

2 Policy and verification extension
Add tool permissions, path restrictions, completion criteria, tests, linters, schemas, and task-specific sensors.

3 Durable memory service
Separate conversation history, long-term facts, active task state, and reusable lessons. Retrieve only relevant items before an agent run.

4 SDK supervisor
Own coordinator, research, execution, and independent verification sessions. Pass concise artifacts rather than entire transcripts.

5 Background runtime
Add scheduling, cancellation, timeouts, retries, heartbeats, work queues, and crash recovery.

6 Sandboxing
Introduce enforceable execution boundaries before unattended agents receive broad shell, filesystem, network, or credential access.

Target mental model:

```
Pi core loop
 inside AgentSession
  inside your SDK supervisor
   inside a safe execution environment
```

10. Practical next step

The best first implementation is a single extension that acts as a harness instrumentation and verification layer. It should subscribe to agent and tool events, write structured run records, classify failure causes, and enforce one or two measurable completion checks. This will reveal where the present Pi behaviour actually falls short in real use before a multi-agent supervisor adds complexity.

After several real sessions, use the captured failures to decide whether the next missing component is memory, permissions, verification, orchestration, or a specialist worker. This turns harness development into an evidence-driven ratchet rather than an attempt to design every capability upfront.

Source index

All entries below are clickable in PDF readers that support standard URI link annotations.

Source	Used for
Official Pi repository	Repository overview, packages, permissions, current project structure.
Coding-agent README	Interactive modes, tools, sessions, compaction, resources, philosophy, and programmatic usage.
CLI entry point - main.ts	Argument parsing, trust, resources, runtime construction, and mode selection.
Interactive mode	Terminal input loop and AgentSession prompt submission.
AgentSession	Prompt preflight, extensions, persistence, retry, compaction, and continuation.
Agent class	Agent state, lifecycle, event processing, and core-loop invocation.
Core agent loop	Nested tool/queue loop, provider streaming, tool execution, and stop conditions.
pi-agent-core documentation	Message flow, event order, tool execution, queues, and low-level API.
pi-ai documentation	Provider abstraction, streaming, auth, schemas, usage, and costs.
System prompt builder	System prompt, context files, tool guidance, skills, and working directory.
Extension API types	Extension hooks, tools, commands, providers, events, and UI surface.
SDK documentation	Embedding Pi, AgentSession, AgentSessionRuntime, custom tools, and multi-session integration.
Harness Engineering video	Wider conceptual framing for harness primitives and reliable agentic systems.

Final conclusion

Pi already supplies the two innermost layers: the core loop and a durable coding-agent session. The highest-value work is outside those layers: policies, verification, memory, supervision, background execution, and sandboxing. Preserve the small loop; build power around it.